

Python

Summary Session 13





Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Экспресс-опрос



Что такое исключения? Назовите примеры исключений



Исключения (Exceptions)

Это события, возникающие во время выполнения программы, которые сигнализируют об ошибочной ситуации, но могут быть обработаны, чтобы программа продолжила выполнение



Примеры исключений



Исключение	Причина возникновения	Пример возникновения
ZeroDivisionError	деление на ноль	<code>print(10 / 0)</code>
ValueError	ошибка значения	<code>int("abc")</code>
KeyError	обращение к несуществующему ключу словаря	<code>info = {"a": 1}</code> <code>print(info["b"])</code>



Обработка исключений помогает



Повысить стабильность программы



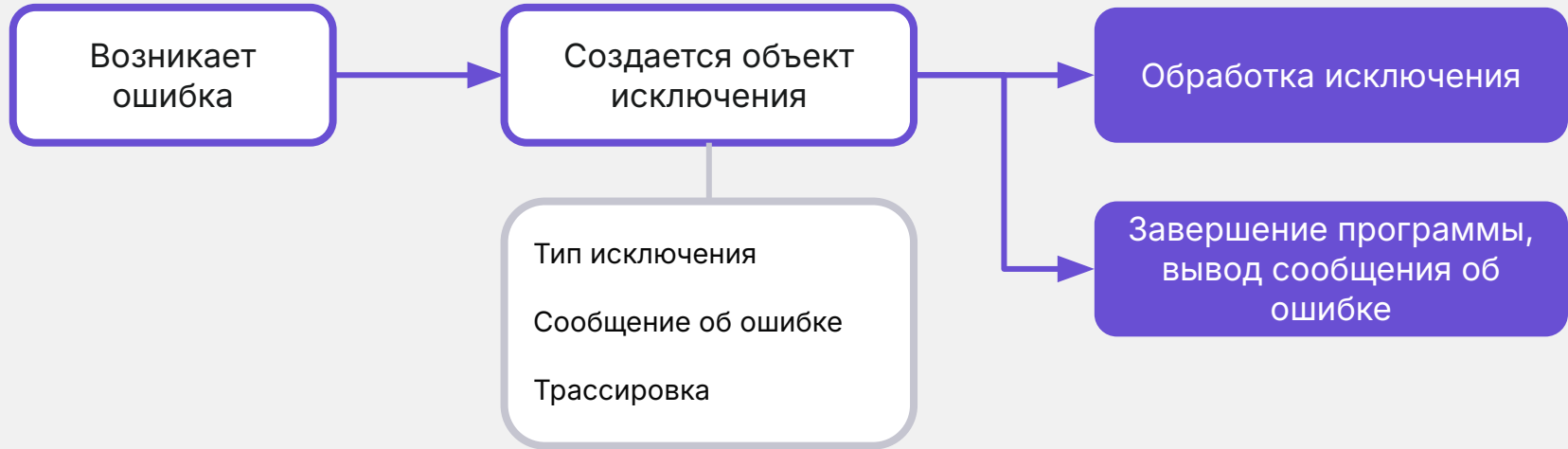
Улучшить пользовательский опыт



Упростить процесс отладки



Реакция Python на ошибки





Пример



Код

```
print(10 / 0)  # Ошибка: ZeroDivisionError
```

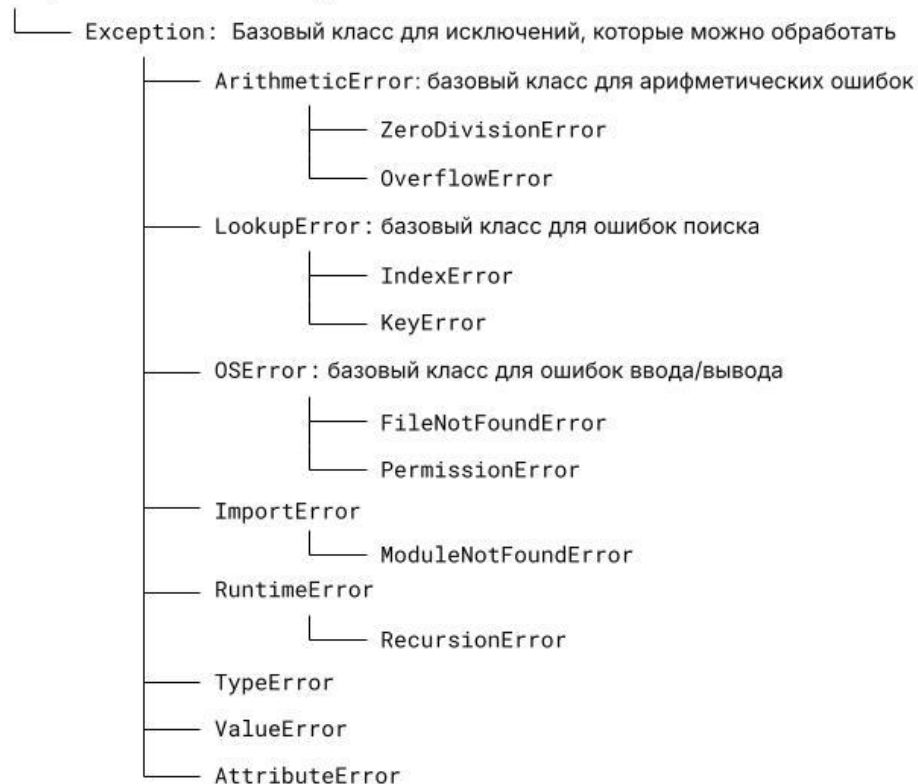
Результат

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
Cell In[68], line 1  
----> 1 print(10 / 0)  
  
ZeroDivisionError: division by zero
```

Основные ветви иерархии



BaseException: Базовый класс для всех исключений.





Назначение обработки исключений



Обработка исключений позволяет перехватывать ошибки, возникающие во время выполнения программы, чтобы предотвратить её аварийное завершение.



Экспресс-опрос

?

Какая конструкция используется для обработки исключений?



Конструкция try-except



Синтаксис

```
try:  
    # Код, который может вызвать  
    исключение  
except ExceptionType:  
    # Код, который выполнится в случае  
    исключения
```

Пояснение

try-except – базовый механизм обработки исключений

ExceptionType – тип исключения, который вы хотите обработать



Пример



```
try:
    result = 10 / 0 # Возникает ZeroDivisionError
    # Следующий код не достигим при ошибке
    print("Деление выполнено успешно!")
    print(result)
except ZeroDivisionError:
    print("Ошибка: деление на ноль!")
```



Экспресс-опрос

?

В каком порядке в Python обрабатываются исключения?



Выбор обработчика исключений

1

Логика

Python проверяет обработчики except сверху вниз. Первый совпавший блок except выполняется, а остальные игнорируются

Пример

```
try:
    user_input = input("Введите ненулевое число: ")
    # Может возникнуть ValueError
    result = 10 / int(user_input) # Может
    возникнуть ZeroDivisionError
    print(f"Результат: {result}")
except ValueError:
    print("Некорректное значение!") #
    Пропускается, если ошибка не ValueError
except ZeroDivisionError:
    print("Ошибка деления на ноль!") # Этот блок
    выполнится, если ошибка ZeroDivisionError
```



Выбор обработчика исключений

2

Логика

Если нет подходящего обработчика,
программа завершает работу с ошибкой

Пример

```
try:
    result = 10 / 0
    # Возникает ZeroDivisionError
except IndexError:
    print("Индекс вне диапазона!")
    # Пропускается
except KeyError:
    print("Ключ не существует!")
    # Пропускается
```



Выбор обработчика исключений

3

Логика

Общий обработчик Exception перехватывает все типы исключений

Пример

```
try:
    result = 10 / 0
    # Возникает ZeroDivisionError
except IndexError:
    print("Индекс вне диапазона!")
    # Пропускается
except KeyError:
    print("Ключ не существует!")
    # Пропускается
except Exception:
    print("Ошибка обработки!")
    # Выполнится, так как `Exception`
    перехватывает `ZeroDivisionError`
```



Выбор обработчика исключений

4

Логика

Exception нужно ставить в конце

Пример

```
try:
    result = 10 / 0
except Exception:
    # Перехватывает всё, обработчики ниже не
    # сработают
    print("Ошибка обработки!")
except ZeroDivisionError:
    print("Ошибка деления на ноль!")
    # Этот код недостижим
```



Экспресс-опрос



Можно ли в Python обрабатывать несколько исключений в одном блоке?



Обработка нескольких исключений в одном блоке



Python позволяет обрабатывать несколько типов исключений в одном блоке `except`, используя кортеж из типов исключений.



Обработка нескольких исключений в одном блоке



Синтаксис

```
try:  
    # Код, который может вызвать исключение  
except (ExceptionType1, ExceptionType2):  
    # Код, который выполнится в случае  
    # любого из указанных исключений
```

Пример

```
while True:  
    try:  
        user_input = input("Введите ненулевое  
число: ")  
        result = 10 / int(user_input)  
        print(f"Результат: {result}")  
        break  
    # Выход из цикла, если ошибок не было  
    except (ZeroDivisionError, ValueError):  
    # Просим пользователя повторить ввод  
        print("Ошибка! Введите корректное  
ненулевое число.")
```



Сообщение об ошибке



Пример

```
try:
    # Попытка преобразования строки в число
    number = int("abc")
except (ZeroDivisionError, ValueError) as e:
    # Выводим текст исключения
    print(f"Произошла ошибка: {e}")
```




Экспресс-опрос

?

Как работает конструкция try-except-else?



Конструкция try-except-else



Если исключение не возникло в блоке `try`, будет выполнен код в блоке `else`



Конструкция try-except-else



Синтаксис

```
try:
# Код, который может вызвать исключение
except ExceptionType:
# Код, который выполнится при возникновении
исключения
else:
# Код, который выполнится, если исключение
НЕ возникло
```

Пример

```
try:
# Преобразование строки в число
    number = int(input("Введите число: "))
except ValueError:
# Обработка некорректного ввода
    print("Ошибка! Введите корректное число.")
else:
# Выполняется только если исключения не было
    print(f"Вы ввели число: {number}")
```



Экспресс-опрос



Какие есть правила использования конструкции try-except-else?



Использование try-except-else

Для **разделения логики программы**

- Улучшает читаемость программы
- Снижает количество дополнительных операций в случае ошибки

Важно: Включайте в блок try только код, **который может вызвать исключение**

- Это улучшает читаемость
- Упрощает отладку
- Позволяет точнее обрабатывать исключения



Пример использования try-except-else



```
try:
    # Проверка числа на чётность
    number = int(input("Введите число: "))
except ValueError:
    # Обработка некорректного ввода
    print("Ошибка! Введите корректное число.")
else:
    # Выполняется только если число успешно введено
    if number % 2 == 0:
        print(f"{number} – чётное число.")
    else:
        print(f"{number} – нечётное число.")
```



Экспресс-опрос

?

Как называется блок, который выполняется всегда, независимо от того, возникло ли исключение в блоке `try`?



Конструкция try-except-finally

`finally` – это блок, который выполняется всегда, независимо от того, возникло ли исключение в блоке `try`.

Используется для завершения операций, которые должны быть выполнены в любом случае.



Конструкция try-except-finally



Синтаксис

```
try:
# Код, который может вызвать исключение
except ExceptionType:
# Код, который выполнится при возникновении
исключения
finally:
# Код, который выполнится всегда,
независимо от исключений
```

Пример

```
try:
# Преобразуем строку в число
    number = int(input("Введите число: "))
    result = 10 / number
except ValueError:
# Обработка некорректного ввода
    print("Ошибка! Введите корректное число.")
except ZeroDivisionError:
# Обработка деления на ноль
    print("Ошибка! Деление на ноль.")
finally:
# Этот код выполнится в любом случае
    print("Завершение программы.")
```



Конструкция try-except-else-finally



Объединяет все блоки обработки исключений и завершения:

- `try`: Выполняется код, который может вызвать исключение
- `except`: Выполняется, если в блоке `try` возникает исключение
- `else`: Выполняется, если в блоке `try` не возникло исключений
- `finally`: Выполняется в любом случае, независимо от того, было исключение или нет



Пример



```
try:
    # Попытка преобразовать строку в число и выполнить деление
    number = int(input("Введите число: "))
    result = 10 / number
except ValueError:
    # Обработка некорректного ввода
    print("Ошибка: введено некорректное значение.")
except ZeroDivisionError:
    # Обработка деления на ноль
    print("Ошибка: деление на ноль.")
else:
    # Выполняется только если исключений не было
    print(f"Результат деления: {result}")
finally:
    # Завершающие действия
    print("Программа завершена.")
```



Экспресс-опрос

?

Что делает ключевое слово raise?



Ключевое слово raise



Позволяет вручную вызвать исключение в любой части программы, чтобы указать на возникшую проблему.



Ключевое слово raise



Синтаксис

```
raise ExceptionType("Сообщение об ошибке")
```

ExceptionType – тип исключения,
который вы хотите вызвать

Пример

```
number = -1
if number < 0:
    raise ValueError("Число не может быть  
отрицательным")
```



Экспресс-опрос

?

Что нужно сделать после использования raise?



После использования raise

1. Либо **перехватите исключение** с помощью try-excerpt и обработайте его.
2. Либо **позвольте программе завершиться**, чтобы пользователь увидел информацию об ошибке.



Пример перехвата исключения после использования raise

```
while True:
    try:
        # Ввод числа пользователем
        number = int(input("Введите положительное число: "))
        if number < 0:
            raise ValueError("Число не может быть отрицательным")
        print(f"Вы ввели корректное число: {number}")
        break # Завершаем цикл, если число корректное
    except ValueError as e:
        # Обработка некорректного ввода
        print(f"Ошибка: {e}. Попробуйте снова.")
```



Экспресс-опрос



Как называется процесс записи информации о работе программы, включая ошибки, предупреждения и отладочные сообщения?



Логирование

Это процесс записи информации о работе программы, включая ошибки, предупреждения и отладочные сообщения



Основы логирования

Модуль logging используется в Python для логирования. Он поддерживает 5 уровней логирования

Уровень	Метод	Описание
DEBUG	<code>logging.debug</code>	Отладочные сообщения (для диагностики программы).
INFO	<code>logging.info</code>	Общая информация о работе программы.
WARNING	<code>logging.warning</code>	Предупреждения о потенциальных проблемах.
ERROR	<code>logging.error</code>	Ошибки, но программа продолжает работать.
CRITICAL	<code>logging.critical</code>	Критические ошибки, после которых программа может завершиться.



Пример логирования



Пример

```
import logging

# Вывод различных сообщений
logging.debug("Отладочное сообщение")
logging.info("Информационное
сообщение")
logging.warning("Предупреждение!")
logging.error("Ошибка!")
logging.critical("Критическая ошибка!")
```

Вывод

```
WARNING:root:Предупреждение!
ERROR:root:Ошибка!
CRITICAL:root:Критическая ошибка!
```

По умолчанию logging показывает только сообщения WARNING и выше

Чтобы увидеть DEBUG и INFO, нужно задать `level=logging.DEBUG` в `basicConfig`



Настройки конфигурации лога



В Jupyter Notebook (и Google Colab) после первоначальной настройки конфигурации лога повторные вызовы `basicConfig()` не произведут никакого эффекта. Можно изменять конфигурацию другими функциями или удалять предыдущие конфигурации



Пример логирования



Пример

```
logging.root.removeHandler(logging.root.handlers
[0]) # Удаление обработчика логов
# Настройка логирования (по умолчанию уровень
WARNING и выше)
logging.basicConfig(level=logging.DEBUG)

# Вывод различных сообщений
logging.debug("Отладочное сообщение")
logging.info("Информационное сообщение")
logging.warning("Предупреждение!")
logging.error("Ошибка!")
logging.critical("Критическая ошибка!")
```

Вывод

```
DEBUG:root:Отладочное сообщение
INFO:root:Информационное сообщение
WARNING:root:Предупреждение!
ERROR:root:Ошибка!
CRITICAL:root:Критическая ошибка!
```



Запись логов в файл



Пример

Можно настроить запись логов
не только **в консоль**, но и **в файл**

```
logging.root.removeHandler(logging.root.handlers[0]) #  
Удаление обработчика логов  
  
# Настройка логирования с записью в файл  
logging.basicConfig(filename="app.log", level=logging.INFO)  
  
logging.info("Программа запущена")  
logging.warning("Низкий уровень памяти")  
logging.error("Ошибка подключения к базе данных")
```




Использование логов в обработке исключений

Логирование часто
используется для записи
ошибок



Пример

```
logging.root.removeHandler(logging.root.handlers[0]) #  
Удаление обработчика логов  
  
logging.basicConfig(filename="app.log", level=logging.ERROR)  
  
try:  
    result = 10 / 0  
except ZeroDivisionError as e:  
    logging.error(f"Ошибка: {e}")
```



Экспресс-опрос



Какой модуль позволяет задавать собственный формат сообщений для логирования?



Настройка формата логов



Объяснение

Модуль `logging` позволяет задавать собственный формат сообщений для логирования. Это делает логи более читаемыми и информативными

Синтаксис

`format="..."` в `logging.basicConfig()`



Ключевые плейсхолдеры

Плейсхолдер	Описание
<code>%(asctime)s</code>	Время записи лога в формате YYYY-MM-DD HH:MM:SS
<code>%(levelname)s</code>	Уровень логирования (DEBUG, INFO, WARNING, ERROR, CRITICAL)
<code>%(filename)s</code>	Имя файла, в котором выполняется логирование
<code>%(lineno)d</code>	Номер строки кода, где был вызван лог
<code>%(message)s</code>	Текст самого лог-сообщения



Пример

```
logging.root.removeHandler(logging.root.handlers[0]) #

# Настройка формата логов
logging.basicConfig(
    filename="app.log",
    format="%(asctime)s - %(filename)s - %(lineno)d - %(levelname)s -
%(message)s",
    level=logging.DEBUG
)

logging.debug("Это отладочное сообщение")
logging.info("Информационное сообщение")
logging.warning("Предупреждение")
logging.error("Ошибка")
logging.critical("Критическая ошибка")
```



Экспресс-опрос

?

Что такое файловая система?



Файловая система

Это способ хранения и организации данных на диске компьютера. Это структура из файлов и папок (директорий), которые можно читать, изменять, удалять или перемещать.



Пример структуры файловой системы

```
/home/user/  
├── PycharmProjects/  
│   ├── project1/  
│   │   ├── data  
│   │   │   └── file.txt  
│   │   └── main.py  
│   └── project2/  
│       └── main.py  
├── documents/  
└── downloads/
```




Экспресс-опрос



Что такое абсолютный и относительный путь?



Абсолютный путь

Это полный путь от корневой директории до файла или папки. Он указывает точное местоположение объекта в файловой системе.



Пример абсолютного пути



#Абсолютный путь в Linux/macOS

```
absolute_path = "/home/user/documents/file.txt"
```

#Абсолютный путь в Windows

```
absolute_path = "C:\\Users\\User\\Documents\\file.txt"
```



Относительный путь

Это путь к файлу или папке относительно текущей рабочей директории. Не показывает путь до корневой директории



Пример относительного пути



#текущая директория /home/user/PycharmProjects/project1/data

```
./file.txt
```

#указывает на файл /home/user/PycharmProjects/project1/data/file.txt

```
../file.txt
```

#указывает на файл /home/user/PycharmProjects/project1/file.txt



Экспресс-опрос

?

Для чего лучше использовать абсолютный, а для чего относительный пути?



Использование

Абсолютный путь

- Когда нужно работать с файлами независимо от текущей директории
- Когда путь к файлу фиксирован (например, конфигурационные файлы)

Относительный путь

- Когда работа с файлами локализована в пределах текущей директории
- Для перемещаемых скриптов



Специальные обозначения для пути

Текущая директория

Знак	Пояснение
.	Указывает на текущую директорию, где находится программа во время выполнения. Используется для построения относительных путей

Windows

MacOS

Linux

```
./file.txt
```

#файл file.txt в текущей директории



Специальные обозначения для пути

Родительская директория

Знак	Пояснение
..	Указывает на директорию на уровень выше (родительскую)

Windows

MacOS

Linux

```
../file.txt
```

#файл file.txt в родительской директории

```
../../file.txt
```

#файл в директории на два уровня выше



Разделитель в пути

Специальные обозначения для пути

Знак	Пояснение
/	В Linux и macOS используется для указания вложенных директорий

macOS

Linux

```
/home/user/file.txt
```

#абсолютный путь



Специальные обозначения для пути

Разделитель в пути

Знак	Пояснение
\\	Только в Windows

Windows

```
C:\\Users\\User\\Documents\\file.txt
```

#абсолютный путь



Специальные обозначения для пути

Домашняя директория

Знак	Пояснение
~	Указывает на домашнюю директорию пользователя (Linux и macOS)

MacOS

Linux

```
~/documents/file.txt
```

#путь к файлу в директории documents домашней папки



Экспресс-опрос



Какой модуль предоставляет инструменты для работы с операционной системой, включая файловую систему?



Модуль os



Модуль **os** предоставляет инструменты для работы с операционной системой, включая файловую систему

#Чтобы использовать модуль, нужно его импортировать
`import os`



Экспресс-опрос

?

Что делают методы `os.getcwd()` и `os.chdir(path)`?



`os.getcwd()`

Это метод, который используется для определения текущей рабочей директории



Пример получения названия текущей директории

```
# Получаем текущую рабочую директорию
current_dir = os.getcwd()
print(f"Текущая директория: {current_dir}")
```



`os.chdir(path)`

Это метод, который используется для перехода в другую директорию



Пример изменения текущей директории

```
# Переход в директорию с использованием относительного пути
os.chdir("./subdirectory") # Ошибка, если директории нет!
print(f"Текущая директория: {os.getcwd()}")
```

```
# Переход в родительскую директорию
os.chdir("..")
print(f"Текущая директория после перехода: {os.getcwd()}")
```

```
# Переход в директорию с использованием абсолютного пути
os.chdir("/home/user/documents") # Замените путь на существующий
print(f"Текущая директория: {os.getcwd()}")
```



Экспресс-опрос

?

Как проверить существование файла или директории?



Проверка существования объекта



Для проверки существования файла или директории используется функция `os.path.exists(path)`



Пример проверки существования файла и директории

```
# Проверка существования файла
file_path = "example.txt"
# или
# file_path = "example_folder/example.txt"
if os.path.exists(file_path):
    print(f"Файл '{file_path}' существует.")
else:
    print(f"Файл '{file_path}' не найден.")

# Проверка существования директории
directory_path = "example_folder"
# или
# directory_path = "parent_folder/child_folder"
if os.path.exists(directory_path):
    print(f"Директория '{directory_path}' существует.")
else:
    print(f"Директория '{directory_path}' не найдена.")
```



`os.listdir(path)`

Это метод, который возвращает список файлов и папок в указанной директории



Пример получения содержимого директории

```
# Список содержимого текущей директории
contents = os.listdir(".")
print("Содержимое текущей директории:", contents)

# Список содержимого указанной директории
specific_dir = "parent_folder"
if os.path.exists(specific_dir):
    print(f"Содержимое директории '{specific_dir}':",
os.listdir(specific_dir))
```




Экспресс-опрос

?

Какие методы позволяют создать новую директорию?



Создание новой директории

`os.mkdir(path)`

- Создаёт одну директорию
- Вызывает ошибку, если директория уже существует

`os.makedirs(path)`

- Создаёт директорию вместе с родительскими, если их ещё нет
- Вызывает ошибку, если директория уже существует, но с использованием параметра `exist_ok=True` ошибка не возникает



Пример создания директории и вложенных директорий

```
dir_path = "example_folder"
if not os.path.exists(dir_path):
    # Создание одной директории
    os.mkdir("example_folder")
    print(f"Директория '{dir_path}' создана.")
else:
    print(f"Директория '{dir_path}' существует.")

# Создание вложенных директорий
os.makedirs("parent_folder/child_folder", exist_ok=True)
print("Вложенные директории 'parent_folder/child_folder' созданы.")
```



`open(path, mode)`

Это функция, которая создает файл.
Она не является частью модуля `os`

Пример создания файла



```
# Создание нового пустого файла
file_name = "example.txt"
open(file_name, "w").close()
print(f"Файл '{file_name}' создан или перезаписан.")
```



Экспресс-опрос



Как можно проверить, является ли объект файлом или директорией?



`os.path.isfile(path)`

Это метод, который используется для проверки, является ли указанный путь файлом



Пример определения типа объекта: файл или нет

```
file_path = "example.txt"
# Проверяем, существует ли объект
if os.path.exists(file_path):
    # Проверяем, является ли файлом
    if os.path.isfile(file_path):
        print(f"'{file_path}' существует и это файл.")
    else:
        print(f"'{file_path}' существует, но это не файл.")
else:
    print(f"'{file_path}' не существует.")
```




`os.path.isdir`

Это метод, который позволяет проверить, является ли путь директорией



Пример определения типа объекта: файл или директория

```
import os

path = "example.txt"
# path = "parent_folder"

# Проверяем, является ли файлом
if os.path.isfile(path):
    print(f"'{path}' существует и это файл.")
# Проверяем, является ли директорией
elif os.path.isdir(path):
    print(f"'{path}' существует и это директория.")
```



Экспресс-опрос

?

Какая функция используется для переименования файлов или директорий?



`os.rename(src, dst)`

Эта функция используется для
переименования файлов или директорий



Пример переименования

Переименование файла

```
old_file_name = "example.txt"
new_file_name = "renamed.txt"
if os.path.exists(old_file_name):
    os.rename(old_file_name, new_file_name)
    print(f"Файл переименован в '{new_file_name}'.")
else:
    print(f"Файл '{old_file_name}' не найден.")
```

Переименование директории

```
old_dir_name = "example_folder"
new_dir_name = "renamed_folder"
if os.path.exists(old_dir_name):
    os.rename(old_dir_name, new_dir_name)
    print(f"Директория переименована в '{new_dir_name}'.")
else:
    print(f"Директория '{old_dir_name}' не найдена.")
```



Экспресс-опрос



Можно ли с помощью `os.remove(path)` удалить пустую директорию?



`os.remove(path)`, `os.rmdir(path)`

Для удаления файлов используется функция `os.remove(path)`, а для удаления пустых директорий – `os.rmdir(path)`



Пример удаления

```
# Удаление файла
file_to_delete = "renamed.txt"
if os.path.exists(file_to_delete):
    os.remove(file_to_delete)
    print(f"Файл '{file_to_delete}' удалён.")
else:
    print(f"Файл '{file_to_delete}' не найден, удаление невозможно.")

# Удаление пустой директории
empty_dir_to_delete = "renamed_folder"
if os.path.exists(empty_dir_to_delete):
    os.rmdir(empty_dir_to_delete)
    print(f"Пустая директория '{empty_dir_to_delete}' удалена.")
else:
    print(f"Директория '{empty_dir_to_delete}' не найдена или не пуста.")
```




Экспресс-опрос

?

Как можно разделить путь на компоненты?



Разделение пути на компоненты



Функция	Назначение
<code>os.path.split(path)</code>	Разделяет путь на две части: директорию и имя файла
<code>os.path.basename(path)</code>	Возвращает только имя файла или папки
<code>os.path.dirname(path)</code>	Возвращает только директорию без имени файла



Пример получения компонент пути

```
path = "/subdirectory/example.txt"

# Разделение пути на директорию и файл
directory, file = os.path.split(path)
print(f"Директория: {directory}, Файл: {file}")

# Получение только имени файла
print(f"Имя файла: {os.path.basename(path)}")

# Получение только директории
print(f"Директория: {os.path.dirname(path)}")
```



Экспресс-опрос



Можно ли преобразовать относительный путь в абсолютный?



`os.path.abspath(path)`

Это метод, который преобразует
относительный путь в абсолютный

Пример получения абсолютного пути



```
# Относительный путь
relative_path = "example.txt"

# Преобразование в абсолютный путь
absolute_path = os.path.abspath(relative_path)
print(f"Абсолютный путь: {absolute_path}")
```



`os.path.join()`

Это метод, который используется для объединения нескольких компонентов пути к файлам и директориям



Пример соединения путей

```
# Объединение нескольких компонентов
current_dir = os.getcwd()
sub_dir = "docs"
file_name = "data.txt"
full_path = os.path.join(current_dir, sub_dir, file_name)
print(f"Путь: {full_path}")
```




Экспресс-опрос

?

Какая функция позволяет рекурсивно обойти директорию и её содержимое, включая файлы и поддиректории?



os.walk

Это функция, которая позволяет рекурсивно обойти директорию и её содержимое, включая файлы и поддиректории



Пример рекурсивного обхода директории

```
# Рекурсивный обход директории
for root, dirs, files in os.walk("."):
    print(f"Текущая директория: {root}")
    print(f"Поддиректории: {dirs}")
    print(f"Файлы: {files}")
    print("-" * 50)
```



Пример использования: поиск файлов с нужным расширением

```
# Поиск всех .txt файлов
for root, dirs, files in os.walk("."):
    for file in files:
        if file.endswith(".txt"):
            print(f"Найден файл: {os.path.join(root, file)}")
```



Экспресс-опрос



Какой модуль предоставляет доступ к функциям и переменным, которые используются и поддерживаются интерпретатором Python?



Модуль sys

Модуль `sys` предоставляет доступ к функциям и переменным, которые используются и поддерживаются интерпретатором Python. Этот модуль позволяет взаимодействовать с окружением Python, управлять аргументами командной строки, путями поиска модулей, стандартным вводом и выводом

#Чтобы использовать модуль, нужно его импортировать
`import sys`





Скрипт

Это исполняемый файл с кодом



Экспресс-опрос

?

Что такое `sys.argv`?



sys.argv

Это список аргументов для скрипта

- Первый элемент списка (`sys.argv[0]`) всегда имя скрипта
- Остальные элементы — собственно, аргументы

Пример работы с аргументами



```
# Вывод всех аргументов командной строки
print("Все аргументы:", sys.argv)
# Работа с первым аргументом (если он передан)
if len(sys.argv) > 1:
    print(f"Переданный аргумент: {sys.argv[1]}")
else:
    print("Аргументы не переданы.")
```



Запуск python файла через терминал

```
# Замените script.py на название вашего файла  
python script.py argument1
```

Вывод

```
(.venv) tanya@tanya-nitro:~/PycharmProjects/pythonProgramItch$ python script.py argument1  
Все аргументы: ['script.py', 'argument1']  
Переданный аргумент: argument1
```



Обработка нескольких аргументов



Пример

```
import sys

if len(sys.argv) > 1:
    for i, arg in enumerate(sys.argv[1:],
start=1):
        print(f"Аргумент {i}: {arg}")
else:
    print("Аргументы не переданы.")
```

Запуск через терминал

```
python script.py arg1 arg2 arg3
```



Экспресс-опрос

?

Для чего используются аргументы командной строки?



Практическое применение `sys.argv`



Аргументы командной строки часто используются для передачи данных в скрипт, делая программу более гибкой

Пример: вывод содержимого директории



```
import sys
import os

# Проверяем, передан ли аргумент
if len(sys.argv) != 2:
    print("Использование: python script.py <директория>")
    sys.exit(1)

directory = sys.argv[1]

# Проверяем существование директории
if not os.path.isdir(directory):
    print(f"Директория '{directory}' не существует.")
    sys.exit(1)

# Выводим содержимое директории
print(f"Содержимое директории '{directory}':")
for item in os.listdir(directory):
    print(f"- {item}")
```

Запуск через терминал

```
python script.py
/home/user/documents
```



Экспресс-опрос

?

Что делает функция `sys.exit()`?



Функция `sys.exit()` модуля `sys`

Это быстрый способ выйти из программы при возникновении ошибки. Она вызывает исключение `SystemExit`, которое можно перехватить или проигнорировать. Однако если это исключение не обрабатывается, Python завершает выполнение программы



ВОПРОСЫ





ЗАДАНИЕ





Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
try:
    d = {"a": 1, "b": 2}
    print(d["c"])
except IndexError:
    print("Ключ не найден")
except Exception:
    print("Ошибка")
```

- a. Ключ не найден
- b. Ошибка
- c. Программа завершится с ошибкой
- d. Ничего не произойдёт



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
try:
    d = {"a": 1, "b": 2}
    print(d["c"])
except IndexError:
    print("Ключ не найден")
except Exception:
    print("Ошибка")
```

- a. Ключ не найден
- b. Ошибка**
- c. Программа завершится с ошибкой
- d. Ничего не произойдёт



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
try:
    num = int("hello")
except ValueError:
    print("Ошибка преобразования", end="
| ")
finally:
    print("Блок finally")
```

- a. Ошибка преобразования
- b. Блок finally
- c. Ошибка преобразования | Блок finally
- d. Программа завершится с ошибкой



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
try:
    num = int("hello")
except ValueError:
    print("Ошибка преобразования", end="
| ")
finally:
    print("Блок finally")
```

- a. Ошибка преобразования
- b. Блок finally
- c. Ошибка преобразования | Блок finally
- d. Программа завершится с ошибкой



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
try:
    print("Начало")
    raise ValueError
except ZeroDivisionError:
    print("Ошибка деления на ноль")
except Exception:
    print("Общее исключение")
finally:
    print("Блок finally")
```

- a. Начало | Ошибка деления на ноль | Блок finally
- b. Начало | Блок finally
- c. Начало | Общее исключение
- d. Начало | Общее исключение | Блок finally



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
try:
    print("Начало")
    raise ValueError
except ZeroDivisionError:
    print("Ошибка деления на ноль")
except Exception:
    print("Общее исключение")
finally:
    print("Блок finally")
```

- a. Начало | Ошибка деления на ноль | Блок finally
- b. Начало | Блок finally
- c. Начало | Общее исключение
- d. Начало | Общее исключение | Блок finally



Выберите верный вариант ответа

Какие блоки выполняются всегда, независимо от наличия исключений?

- a. try
- b. except
- c. else
- d. finally



Выберите верный вариант ответа

Какие блоки выполняются всегда, независимо от наличия исключений?

- a. try
- b. except
- c. else
- d. finally



Выберите верный вариант ответа

Какой параметр `logging.basicConfig()` задаёт формат вывода логов?

- a. `filename`
- b. `level`
- c. `format`
- d. `log_format`



Выберите верный вариант ответа

Какой параметр `logging.basicConfig()` задаёт формат вывода логов?

- a. `filename`
- b. `level`
- c. `format`
- d. `log_format`



Выберите верный вариант ответа

Какой символ/символы используются для обозначения родительской директории?

- a. \
- b. /
- c. .
- d. ..



Выберите верный вариант ответа

Какой символ/символы используются для обозначения родительской директории?

- a. \
- b. /
- c. .
- d. ..



Выберите верный вариант ответа

Какой метод используется для проверки существования пути?

- a. `os.exists(path)`
- b. `os.file_exists(path)`
- c. `os.path.exists(path)`
- d. `os.is_file(path)`



Выберите верный вариант ответа

Какой метод используется для проверки существования пути?

- a. `os.exists(path)`
- b. `os.file_exists(path)`
- c. `os.path.exists(path)`
- d. `os.is_file(path)`



Выберите верный вариант ответа

Какой метод создаст вложенные директории, если их ещё нет?

- a. `os.mkdir(path)`
- b. `os.makedirs(path, exist_ok=True)`
- c. `os.create_directories(path)`
- d. `os.create_folders(path)`



Выберите верный вариант ответа

Какой метод создаст вложенные директории, если их ещё нет?

- a. `os.mkdir(path)`
- b. `os.makedirs(path, exist_ok=True)`
- c. `os.create_directories(path)`
- d. `os.create_folders(path)`



Выберите верный вариант ответа

Как можно переименовать файл old.txt в new.txt

- a. `os.rename("old.txt", "new.txt")`
- b. `os.move("old.txt", "new.txt")`
- c. `os.change_name("old.txt", "new.txt")`
- d. `os.rename_file("old.txt", "new.txt")`



Выберите верный вариант ответа

Как можно переименовать файл old.txt в new.txt

- a. `os.rename("old.txt", "new.txt")`
- b. `os.move("old.txt", "new.txt")`
- c. `os.change_name("old.txt", "new.txt")`
- d. `os.rename_file("old.txt", "new.txt")`



Выберите верный вариант ответа

Какой метод позволяет получить имя файла из полного пути?

- a. `os.path.split(path)`
- b. `os.path.basename(path)`
- c. `os.path.dirname(path)`
- d. `os.path.getfilename(path)`



Выберите верный вариант ответа

Какой метод позволяет получить имя файла из полного пути?

- a. `os.path.split(path)`
- b. `os.path.basename(path)`
- c. `os.path.dirname(path)`
- d. `os.path.getfilename(path)`



Выберите верный вариант ответа

Какие утверждения о `os.path.join()` верны?

- a. Он корректно соединяет пути независимо от операционной системы
- b. Он позволяет избежать ошибок при ручном объединении путей
- c. Он изменяет текущую директорию
- d. Он создаёт новый файл или папку



Выберите верный вариант ответа

Какие утверждения о `os.path.join()` верны?

- a. Он корректно соединяет пути независимо от операционной системы
- b. Он позволяет избежать ошибок при ручном объединении путей
- c. Он изменяет текущую директорию
- d. Он создаёт новый файл или папку



Выберите верный вариант ответа

Что будет напечатано при выполнении кода?

```
import sys

if len(sys.argv) > 1:
    print(sys.argv[1])
else:
    print("Нет аргументов")
```

Запуск: `python script.py`

- a. `script.py`
- b. `script.py` Нет аргументов
- c. Нет аргументов
- d. Ошибка выполнения



Выберите верный вариант ответа

Что будет напечатано при выполнении кода?

```
import sys

if len(sys.argv) > 1:
    print(sys.argv[1])
else:
    print("Нет аргументов")
```

Запуск: `python script.py`

- a. `script.py`
- b. `script.py Нет аргументов`
- c. **Нет аргументов**
- d. Ошибка выполнения



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА





Задание 1. Поиск файла в системе

Напишите программу, которая:

- Запрашивает у пользователя имя файла.
- Ищет его во всей системе, начиная с текущей директории.
- Выводит полный путь, если файл найден.

Пример ввода

Введите имя файла: `my_script.py`

Пример вывода

Файл найден: `/home/user/projects/my_script.py`



Задание 2

Напишите программу, которая:

- Запрашивает у пользователя путь к директории.
- Находит все пустые папки в этой директории.
- Удаляет их.

Пример ввода

Введите путь к директории: `test_folder`

Пример вывода

Удалены пустые папки: `test_folder/empty1`, `test_folder/empty2`



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ





Домашнее задание

Деление без ошибок

Напишите функцию, которая выполняет деление двух чисел, введенных пользователем, и обрабатывает возможные ошибки.

Пример вывода:

Введите делимое: 345

Введите делитель: 5a

Ошибка: Введено некорректное число.



Домашнее задание

Логирование ошибок

Перенаправьте в предыдущей задаче вывод ошибок в файл `errors.log` в соответствии с форматом ниже.

Пример вывода:

```
2025-02-23 22:38:53,686 - ERROR - test.py - 16 - Ошибка: Введено некорректное число.
```



ВОПРОСЫ





Домашнее задание

Список файлов и папок

Напишите программу, которая принимает путь к директории через аргумент командной строки и выводит:

- Отдельно список папок
- Отдельно список файлов

Пример запуска:

```
python script.py /home/user/documents
```

Пример вывода:

Содержимое директории '/home/user/documents':

Папки:

- folder1
- folder2

Файлы:

- file1.txt
- file2.txt
- notes.docx



ВОПРОСЫ





Заключение

